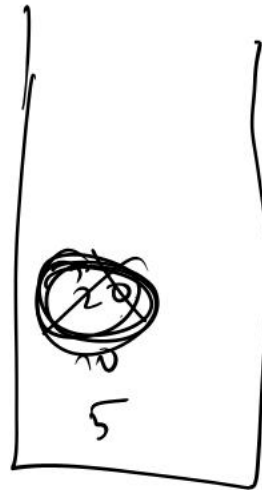
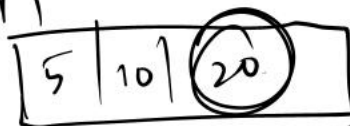


push(5) -

push(10)

push(20)

v = stack.pop()



```
# Stack is a DS, which follows LIFO order
```

```
class Stack :
```

```
    def __init__(self):
```

```
        self.stack = []      # Creating a stack
```

```
    # Push elements into stack
```

```
    def push(self, element) :
```

```
        self.stack.append(element)
```

```
    # Removes the last element in stack
```

```
    def pop(self) :
```

```
        if self.isEmpty() :
```

```
            return 'Stack is Empty!'
```

```
        value = self.stack.pop()
```

```
        return value
```

```
    # Returns the top element in stack
```

```
    def top(self) :
```

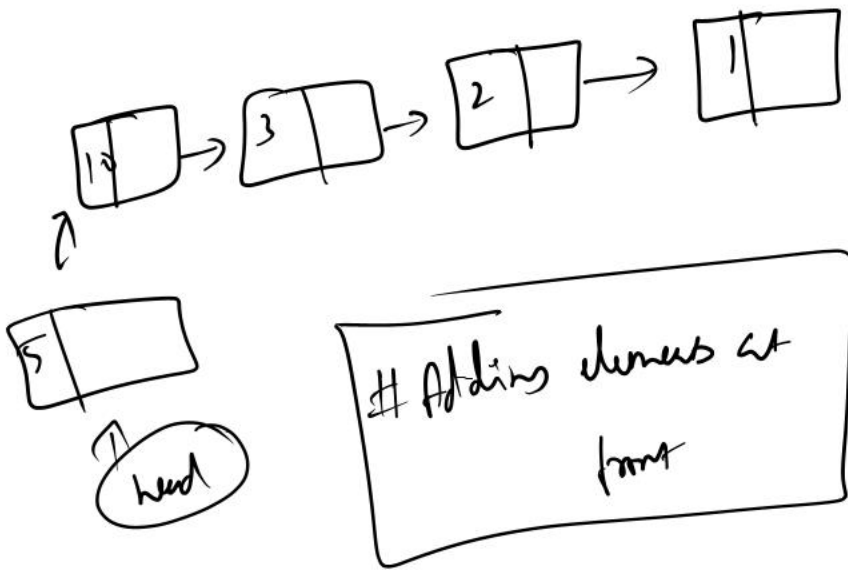
```
        if self.isEmpty() :
```

```
            return 'Stack is Empty!'
```

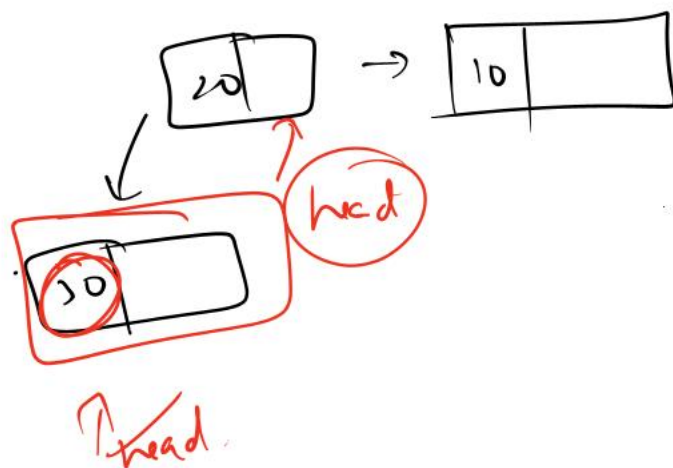
```

    return Stack is Empty!
    return self.stack[-1]
# Returns true if stack is empty, false otherwise
def isEmpty(self) :
    return len(self.stack) == 0

```



push(1)
 push(2)
 push(3)
 push(4)
 push(5)



push(10)

pop()

844. Backspace String Compare

Easy Topics Companies

Given two strings `s` and `t`, return `true` if they are equal when both are typed into empty text editors. `'#'` means a backspace character.

Note that after backspacing an empty text, the text will continue empty.

Example 1:

Input: `s = "ab#c"`, `t = "ad#c"`

Output: `true`

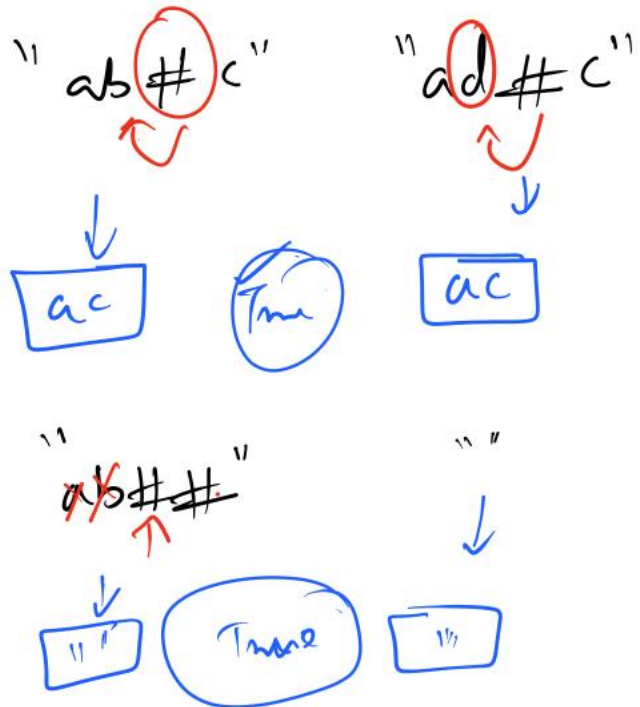
Explanation: Both `s` and `t` become `"ac"`.

Example 2:

Input: `s = "ab##"`, `t = "c#d#"`

Output: `true`

Explanation: Both `s` and `t` become `""`.



`"a#b#c#d#"`

`c` `a` `b` `c` `d` `c`

creating a stack

for c ins:

if '#'

pop()

Empty

Else:

Append c into stack.

```
class Solution:
```

```
    def backspaceCompare(self, s: str, t: str) -> bool:
```

```
        def helper(s) :
```

```
            stack = []
```

```
            for ch in s :
```

```
                if ch == '#' :
```

```
                    if len(stack) != 0 :
```

```
                        stack.pop()
```

```
                else :
```

```
                    stack.append(ch)
```

```
            return stack
```

```
        return helper(s) == helper(t)
```

$\rightarrow \text{F.W} \rightarrow O(n)$

$\rightarrow \text{S.C} \rightarrow O(n)$

682. Baseball Game

Easy

Topics

Companies

You are keeping the scores for a baseball game with strange rules. At the beginning of the game, you start with an empty record.

You are given a list of strings `operations`, where `operations[i]` is the i^{th} operation you must apply to the record and is one of the following:

- An integer `x`.
 - Record a new score of `x`.
- `+`.
 - Record a new score that is the sum of the previous two scores.
- `D`.
 - Record a new score that is the double of the previous score.
- `C`.
 - Invalidate the previous score, removing it from the record.

Return the sum of all the scores on the record after applying all the operations.

Input: ops = ["5", "-2", "4", "C", "D", "9", "+", "+"]

Output: 27

[5, -2, 4, -4, 9, 5, 14)

(27) o/p

class Solution:

def calPoints(self, operations: List[str]) -> int:

stack = []

for elem in operations :

if elem == '+' :

stack.append(stack[-1] + stack[-2])

elif elem == 'D' :

stack.append(stack[-1] * 2)

elif elem == 'C' :

stack.pop()

else :

stack.append(int(elem))

return sum(stack)

T.C -> O(n)

S.C -> O(n)

space to
store the
records.

20. Valid Parentheses

Solved

Easy

Topics

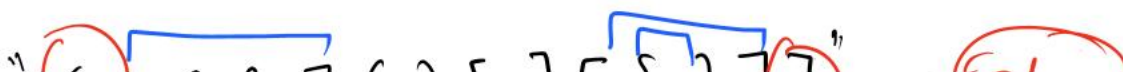
Companies

Hint

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

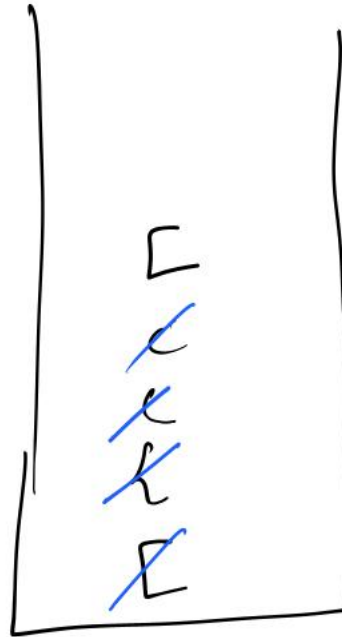
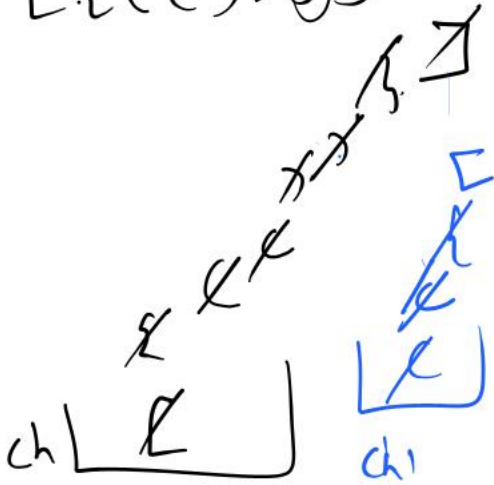
An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.



({ }) () [] { }) → false

"[{ (()) }]"



→ Stack is not Empty

↓
false

class Solution:

def isValid(self, s: str) -> bool:

stack = []

for ch in s :

if ch == '(' or ch == '[' or ch == '{' :

stack.append(ch)

else :

if len(stack) == 0 :

return False

else :

ch1 = stack.pop()

if (ch == ')' and ch1 != '(') or (ch == ']' and ch1 != '[') or (ch == '}' and ch1 != '{') :

return False

return len(stack) == 0

T.C → O(n)

S.C → O(n)

1. 2. 3. 4. 5. 6. 7. 8. 9. 10. 11. 12. 13. 14. 15. 16. 17. 18. 19. 20. 21. 22. 23. 24. 25. 26. 27. 28. 29. 30. 31. 32. 33. 34. 35. 36. 37. 38. 39. 40. 41. 42. 43. 44. 45. 46. 47. 48. 49. 50. 51. 52. 53. 54. 55. 56. 57. 58. 59. 60. 61. 62. 63. 64. 65. 66. 67. 68. 69. 70. 71. 72. 73. 74. 75. 76. 77. 78. 79. 80. 81. 82. 83. 84. 85. 86. 87. 88. 89. 90. 91. 92. 93. 94. 95. 96. 97. 98. 99. 100.

1. 2. 3. 4. 5. 6. 7. 8. 9. 10. 11. 12. 13. 14. 15. 16. 17. 18. 19. 20. 21. 22. 23. 24. 25. 26. 27. 28. 29. 30. 31. 32. 33. 34. 35. 36. 37. 38. 39. 40. 41. 42. 43. 44. 45. 46. 47. 48. 49. 50. 51. 52. 53. 54. 55. 56. 57. 58. 59. 60. 61. 62. 63. 64. 65. 66. 67. 68. 69. 70. 71. 72. 73. 74. 75. 76. 77. 78. 79. 80. 81. 82. 83. 84. 85. 86. 87. 88. 89. 90. 91. 92. 93. 94. 95. 96. 97. 98. 99. 100.

stack

42. Trapping Rain Water

Solved

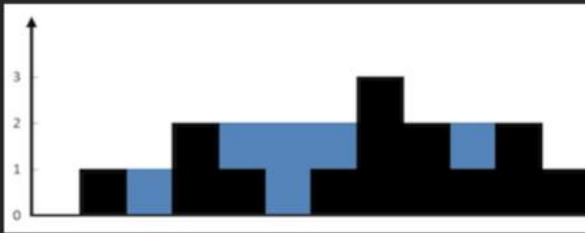
Hard

Topics

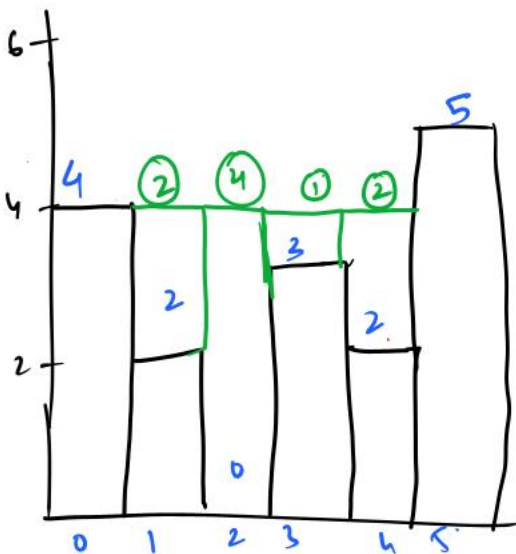
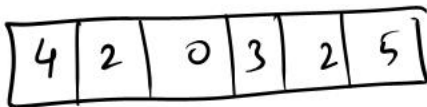
Companies

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example 1:



Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]
Output: 6



Create water = 0

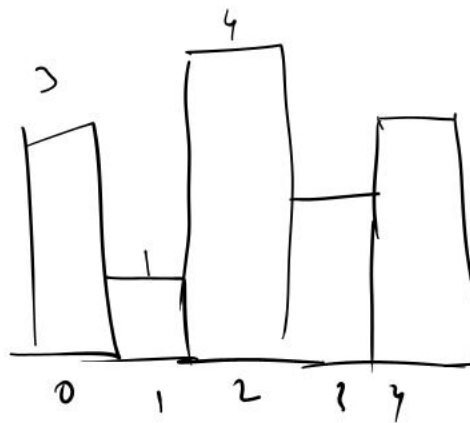
$1 \leq i < n-1$:

Find leftMax $\rightarrow 0 \leq j \leq i$

Find rightMax $\rightarrow i+1 \leq j < n$

water += min(leftMax, rightMax) - height[i]

Return water



class Solution:

def trap(self, height: List[int]) -> int:

water = 0

n ← for i in range(len(height)) :

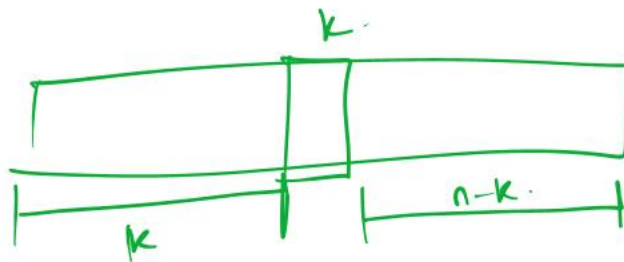
n ← { leftMax = max(height[: i + 1]) } ←

n ← { rightMax = max(height[i :]) } ← n-1

water += min(leftMax, rightMax) - height[i]

return water

T.C
 $n \times n \rightarrow n^2$



$\leftarrow (n) \times k$

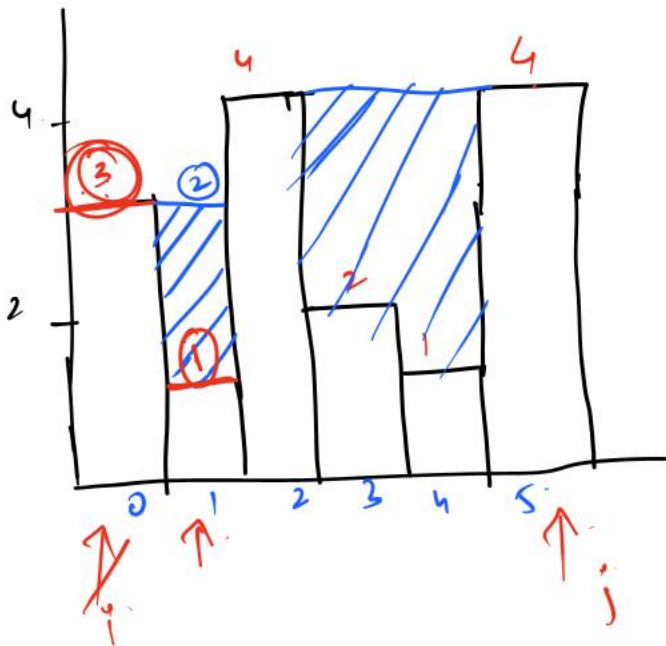
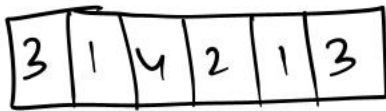
2sec $\rightarrow 10^8$ operations

1sec $\rightarrow 10^9$ operations

$n \rightarrow 2 \times 10^9$

$\rightarrow (2 \times 10^9)^2 \rightarrow 4 \times 10^{18} > 10^8$

↓
 TLE



$i \mid 0 \mid j \mid 5$

leftMax $\mid 3 \mid$ rightMax $\mid 0 \mid$

```
class Solution:
    def trap(self, height: List[int]) -> int:
        water = 0
        i, j = 0, len(height) - 1
        leftMax, rightMax = 0, 0
        while i < j:
            if height[i] < height[j]:
                if height[i] >= leftMax:
                    leftMax = height[i]
                else:
                    water += leftMax - height[i]
                    i += 1
            else:
                if height[j] >= rightMax:
                    rightMax = height[j]
                else:
                    water += rightMax - height[j]
                    j -= 1
        return water
```

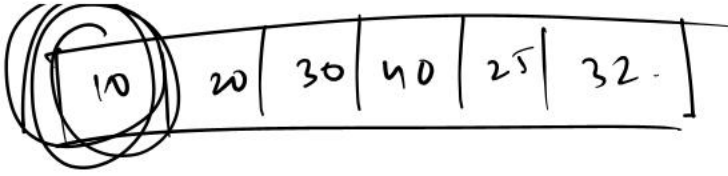


$i \mid 0 \mid j \mid 5$ $\mid 1 \mid 2 \mid 3 \mid 4 \mid 5$

$\rightarrow T.C \rightarrow O(n)$ $S.C \rightarrow O(1)$

Ques

10 20 30 40 25 32



933. Number of Recent Calls

Solved ✓

Easy

Topics

Companies

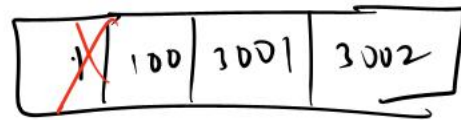
You have a `RecentCounter` class which counts the number of recent requests within a certain time frame.

Implement the `RecentCounter` class:

- `RecentCounter()` Initializes the counter with zero recent requests.
- `int ping(int t)` Adds a new request at time `t`, where `t` represents some time in milliseconds, and returns the number of requests that has happened in the past `3000` milliseconds (including the new request). Specifically, return the number of requests that have happened in the inclusive range `[t - 3000, t]`.

It is guaranteed that every call to `ping` uses a strictly larger value of `t` than the previous call.

Requests → 1, 100, 3001, 3002



range:

$[t - 3000, t]$

3002
3001
100
1
+ [X]

~~-2999 to 1~~

-2900 to 100

~~1, 3001~~
[2, 3002]

ok: [1,
2,
3,
3]

```
class RecentCounter:
```

```
    def __init__(self):
```

```
        self.queue = deque()
```

```
self.queue.append(t)
```

```
def ping(self, t: int) -> int:  
    self.queue.append(t)  
    while self.queue[0] < t - 3000:  
        self.queue.popleft()  
    return len(self.queue)
```